Appendice Tecnica: Analisi Comparativa delle Prestazioni

REST (JSON) vs gRPC (Protobuf) in Sistemi di Telemetria

24 aprile 2026

1 Introduzione

Il presente documento fornisce un'analisi dettagliata e comparativa dei risultati ottenuti durante la fase di benchmarking. L'indagine si focalizza sulla latenza di risposta, sull'efficienza della banda e sulla scalabilità dei protocolli REST e gRPC sotto carichi variabili.

2 Scalabilità Totale e Trend di Latenza

In questa sezione vengono analizzati i trend generali di latenza. Come evidenziato dai grafici in modalità Polling con payload Small, REST risulta superiore dato che la semplicità strutturale di HTTP/1.1 minimizza l'overhead di gestione del protocollo. Al contrario, gRPC soffre della complessità intrinseca dello stack HTTP/2 , che diventa un collo di bottiglia ingiustificato quando il volume di dati è troppo ridotto per beneficiare della compressione binaria.

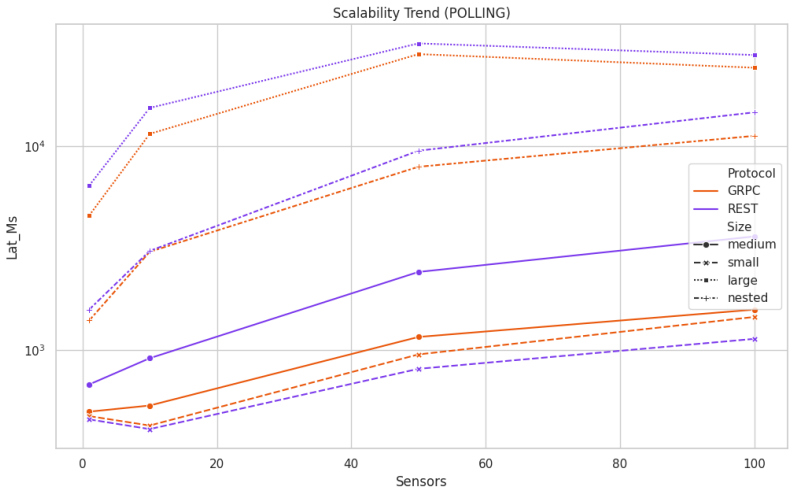


Figura 1: Trend di Scalabilità: Latenza Totale in modalità Polling.

Il paradosso prestazionale per cui gRPC Medium supera REST Small in modalità streaming è la prova definitiva dell'efficienza del protocollo binario. Mentre REST è limitato dall'overhead sintattico del formato JSON e dalla gestione inefficiente dei buffer testuali, gRPC sfrutta la persistenza del canale HTTP/2 e la linearità di Protobuf. Questo permette di trasmettere un volume di informazioni nettamente superiore (200 stringhe dinamiche contro 2 tag statici) in un tempo inferiore.

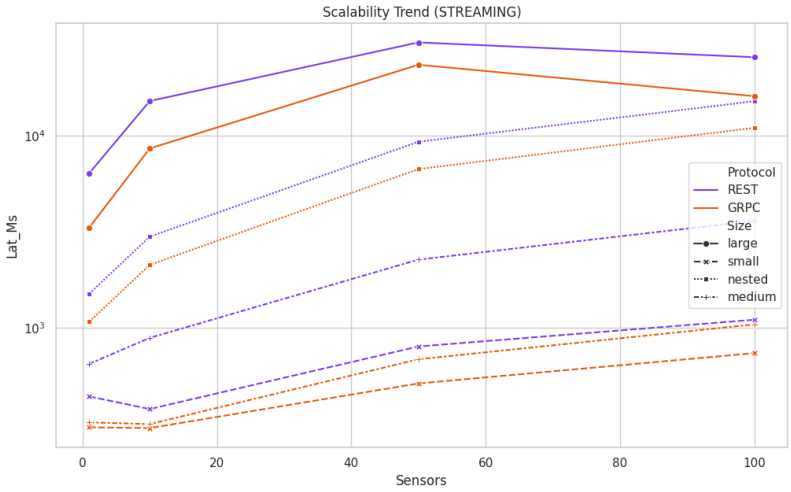


Figura 2: Trend di Scalabilità: Latenza Totale in modalità Streaming.

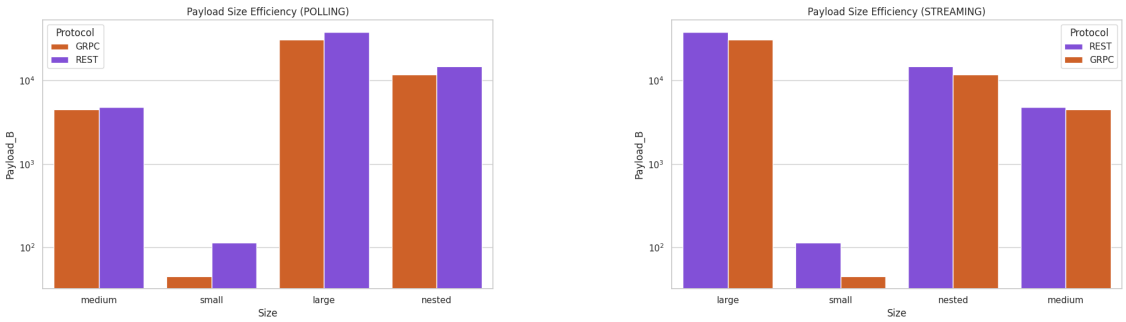
3 Efficienza del Payload e Risparmio di Banda

L'efficienza di gRPC non si riflette solo nella velocità di esecuzione, ma anche in una drastica ottimizzazione delle risorse di rete.

3.1 Analisi del Payload

L'analisi della dimensione del payload (Fig. 3) mette in luce l'efficienza del formato binario rispetto a quello testuale. In questi grafici viene isolato il solo peso dei dati serializzati, escludendo gli header di rete; ciò che emerge è la differenza strutturale tra la verbosità del JSON e la densità informativa di Protocol Buffers.

Nel caso del payload **Small**, lo scarto è netto: mentre REST deve trasportare l'intera struttura sintattica del JSON (parentesi, virgolette e nomi dei campi in chiaro), gRPC riduce il messaggio a pochi byte, codificando le informazioni tramite identificatori numerici minimi. Questa efficienza permette a gRPC di mantenere una densità tale da impacchettare, nel caso **Medium**, 200 stringhe dinamiche in un volume di dati compatto. In definitiva, questi dati dimostrano che il vantaggio di gRPC nasce alla base, massimizzando il *Goodput* (il rapporto tra dati utili e peso totale) prima ancora dell'invio su rete.



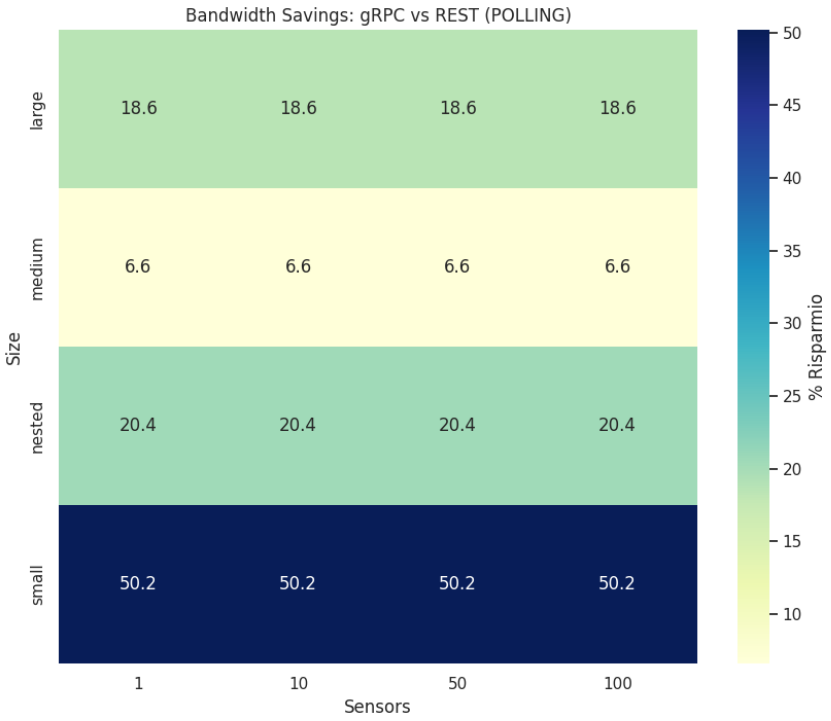
(a) Dimensione Payload (Polling)

(b) Dimensione Payload (Streaming)

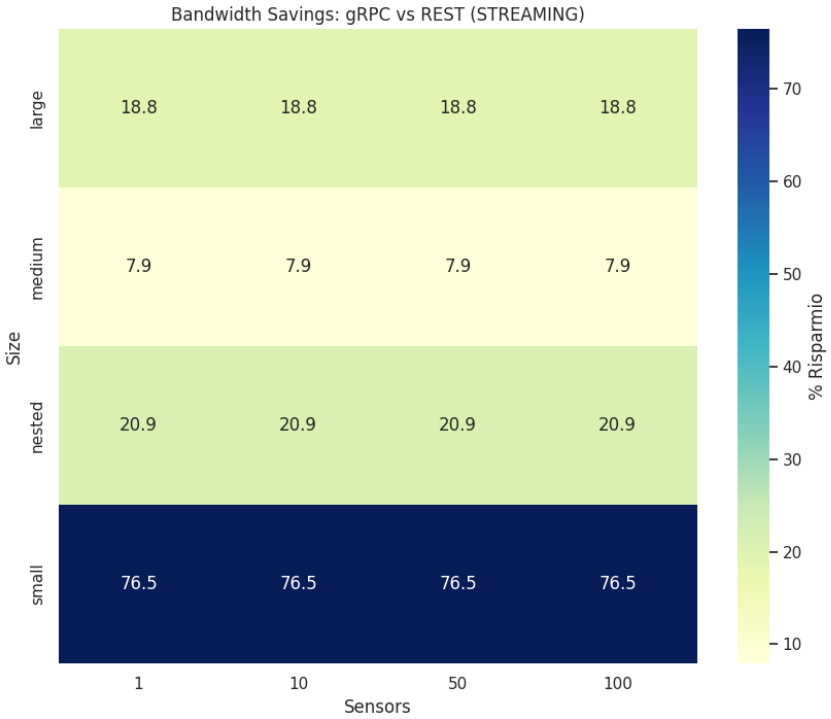
Figura 3: Confronto della dimensione dei messaggi (Scala Logaritmica).

3.2 Analisi del Risparmio Percentuale (Heatmap)

Le heatmap quantificano il risparmio percentuale di banda ottenuto passando da REST a gRPC, certificando un'ottimizzazione che raggiunge il **76.5%** in modalità streaming per i carichi *Small*.



(a) Risparmio Banda - Polling



(b) Risparmio Banda - Streaming

Figura 4: Percentuale di banda risparmiata grazie alla serializzazione binaria.

L'analisi dei dati evidenzia due fenomeni in particolare:

- **Indipendenza dal carico:** Il risparmio percentuale rimane pressoché identico sia con 1 che con 100 sensori (asse orizzontale uniforme), dimostrando che l'efficienza è una proprietà intrinseca del formato e non degrada sotto stress.

- **Il Caso del Payload Medium:** Un dettaglio analitico di particolare rilievo riguarda il payload *Medium*, dove il risparmio percentuale si attesta su valori apparentemente modesti (circa l'8%). Questo fenomeno è dovuto alla natura del dato: il payload *Medium* è composto prevalentemente da **stringhe dinamiche**, un tipo di dato che non beneficia della compressione numerica binaria di gRPC, poiché le stringhe vengono trasmesse come sequenze di byte pressoché identiche in entrambi i protocolli. Tuttavia, è fondamentale notare che, data l'elevata dimensione totale di questo payload, quel "piccolo" 8% di risparmio (derivante esclusivamente dall'eliminazione della sintassi JSON e delle chiavi testuali) si traduce comunque in un risparmio di **Kilobyte assoluti per secondo** molto buono. Ciò dimostra che gRPC rimane più efficiente anche quando il contenuto dei dati non è intrinsecamente binarizzabile.

4 Analisi Granulare: Latency Breakdown

Per ogni test, la latenza è stata scomposta in due componenti fondamentali per identificare l'origine dei guadagni prestazionali:

- **Network & Logic (Barra Chiara):** Rappresenta il tempo speso nella trasmissione dei pacchetti sulla rete e nella gestione del protocollo (header, handshake, multiplexing).
- **Marshalling CPU (Barra Scura):** Rappresenta il tempo computazionale necessario alla serializzazione e deserializzazione dei dati (trasformazione degli oggetti in memoria nel formato di trasporto e viceversa).

4.1 Overview Generale: Il vantaggio strutturale dello Streaming

L'analisi comparativa tra le modalità Polling e Streaming rivela un pattern costante per tutte le tipologie di payload: lo **Streaming rappresenta l'ambiente operativo ideale per gRPC**, permettendogli di abbattere drasticamente i tempi morti della rete e i costi computazionali.

Mentre in modalità **Polling** entrambi i protocolli risentono dell'overhead necessario a stabilire e chiudere continuamente le sessioni, il passaggio allo **Streaming** cambia radicalmente gli equilibri. In questa modalità, gRPC beneficia di una connessione persistente "calda" dove la componente *Network & Logic* si riduce grazie al riuso del tunnel TCP/HTTP2, mentre l'efficienza del *Marshalling* massimizza il rendimento della CPU, che può processare i dati in un flusso binario continuo senza dover ricostruire l'intera struttura del messaggio per ogni pacchetto, come invece deve fare REST con il formato JSON.

4.2 Payload: SMALL

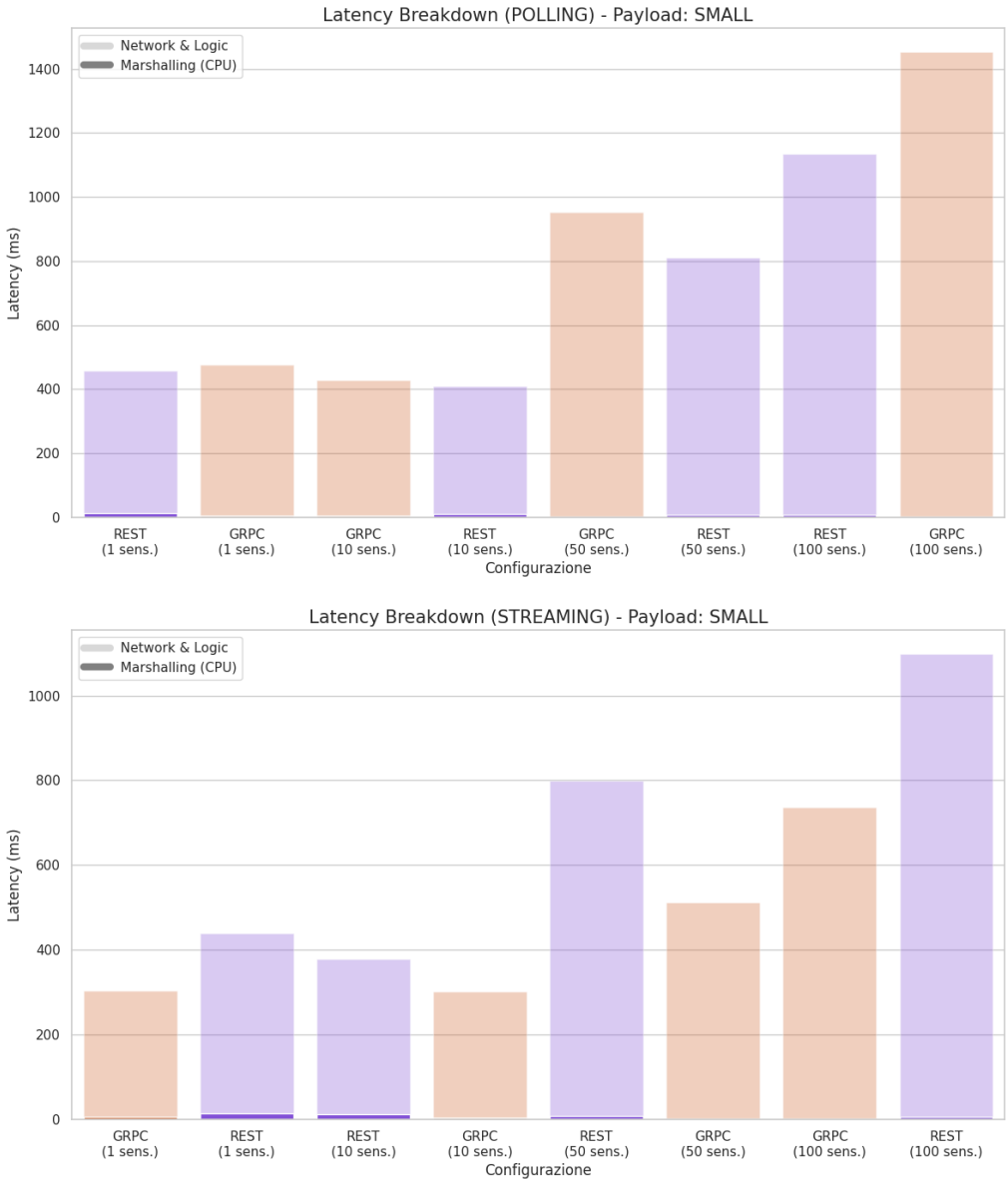


Figura 5: Breakdown Latenza: Payload SMALL (Polling vs Streaming).

Analisi Comparativa: Polling vs Streaming

- **Efficienza del Marshalling:** In modalità Streaming, gRPC riduce il tempo di elaborazione CPU rispetto al Polling. In REST, il costo di parsing rimane costante e oneroso a causa della natura testuale del JSON.
- **Impatto della Rete:** Il passaggio allo Streaming abbatte la componente *Network & Logic* per gRPC grazie alla connessione persistente, mentre in REST la riduzione è marginale.

4.3 Payload: MEDIUM

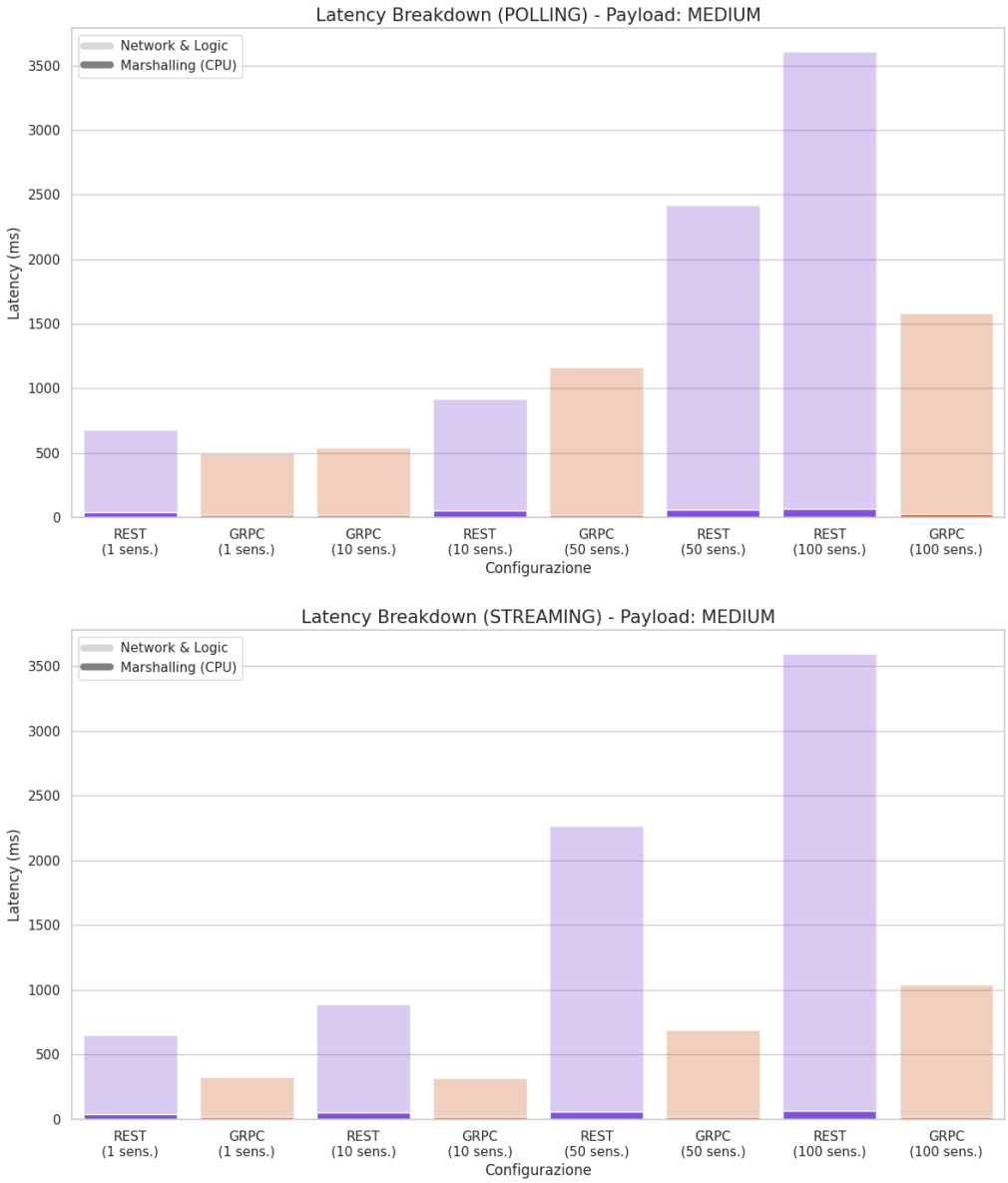


Figura 6: Breakdown Latenza: Payload MEDIUM (Polling vs Streaming).

Analisi Comparativa: Polling vs Streaming

- **Efficienza del Marshalling:** Con l'aumento dei dati (200 stringhe), la velocità del marshaller binario di gRPC in streaming permette di mantenere la barra scura quasi invisibile, mentre in REST il parsing JSON diventa un costo dominante.
- **Impatto della Rete:** gRPC ottimizza drasticamente il flusso, permettendo a 100 sensori in streaming di avere una latenza totale inferiore a 50 sensori in polling.

4.4 Payload: LARGE

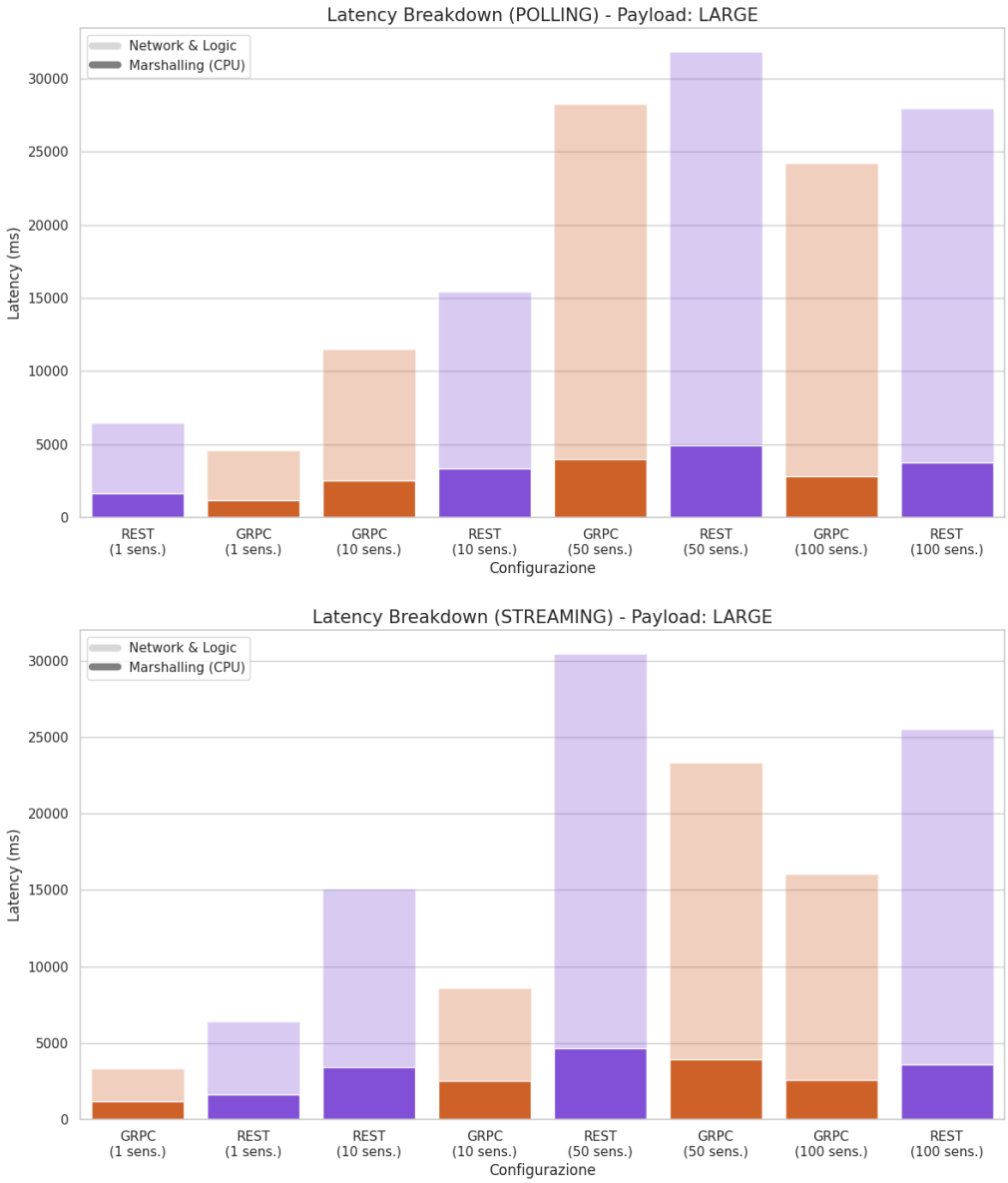


Figura 7: Breakdown Latenza: Payload LARGE (Polling vs Streaming).

Analisi Comparativa: Polling vs Streaming

- **Efficienza del Marshalling:** La binarizzazione dei dati numerici (float) in streaming permette a gRPC di evitare la costosa conversione numero-stringa tipica di REST, riducendo il carico sulla CPU.
- **Impatto della Rete:** Lo streaming binario di gRPC riduce il tempo di transito per grandi volumi di dati, stabilizzando la componente *Network & Logic* anche sotto carico massimo.

4.5 Payload: NESTED

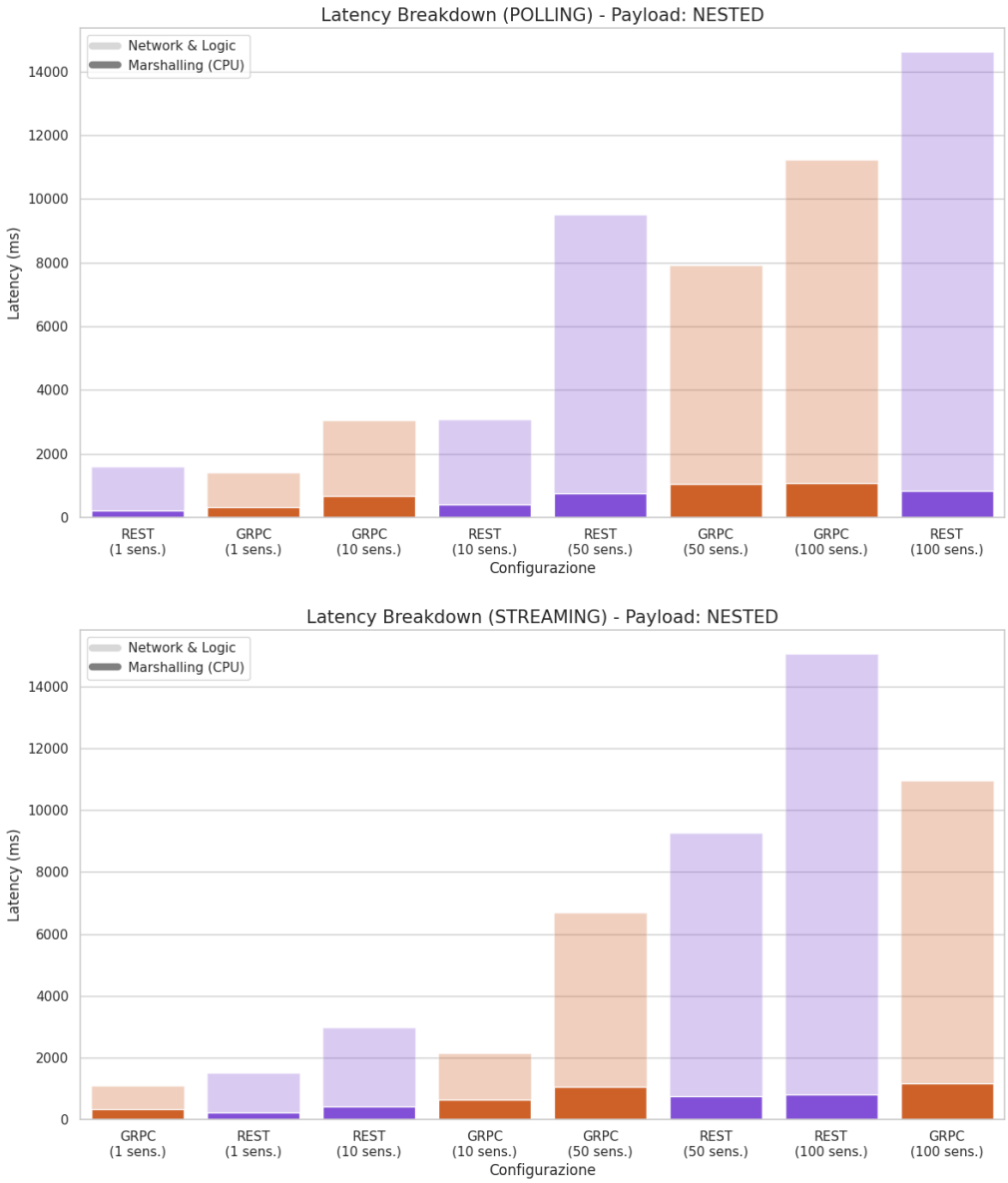


Figura 8: Breakdown Latenza: Payload NESTED (Polling vs Streaming).

Analisi Comparativa: Polling vs Streaming (Payload Nested)

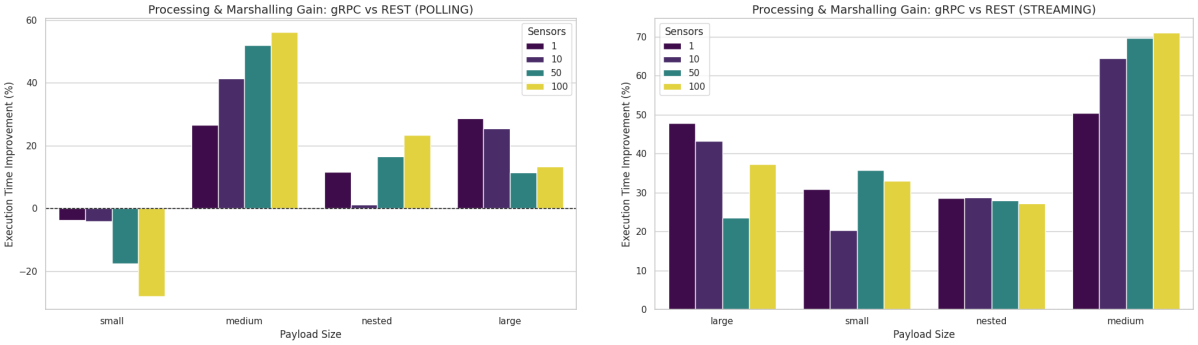
- **Efficienza del Marshalling:** Nel caso *Nested*, si osserva che gRPC impiega un tempo di Marshalling proporzionalmente maggiore rispetto ai payload lineari. Questo è dovuto alla complessità strutturale dell'oggetto che richiede una gestione ricorsiva dei campi. Tuttavia, nonostante questo sforzo computazionale aggiuntivo, il tempo totale rimane nettamente inferiore a quello di REST, poiché gRPC evita la pesante fase di tokenizzazione e parsing testuale necessaria per ricostruire un albero JSON complesso.

- **Impatto della Rete:** Mentre in REST la latenza esplode, gRPC in streaming mantiene una certa stabilità. La connessione persistente permette di "ammortizza-

re” la complessità del dato, garantendo tempi di risposta prevedibili anche con 100 sensori simultanei.

5 Efficienza di Elaborazione e Marshalling Gain

L'analisi del *Marshalling Gain* (Fig. 9) permette di quantificare l'efficienza computazionale di gRPC rispetto a REST, isolando il tempo risparmiato dalla CPU durante le fasi di serializzazione e deserializzazione dei dati. Un valore positivo indica una superiorità di gRPC, mentre un valore negativo evidenzia una maggiore efficienza di REST.



(a) Efficienza Marshalling (Polling)

(b) Efficienza Marshalling (Streaming)

Figura 9: Guadagno percentuale di gRPC rispetto a REST. I valori negativi indicano scenari in cui REST risulta più efficiente.

5.1 Il paradosso del Payload Small in Polling

L'aspetto più controintuitivo dell'analisi emerge osservando il grafico in modalità **Polling** per il payload **Small**. In questo scenario, gRPC registra un guadagno negativo che tocca il **-28%** con 100 sensori: ciò significa che **REST è sensibilmente più veloce di gRPC** nell'elaborare messaggi minimi in modalità stateless.

Questa sottomisura è riconducibile a due fattori tecnici:

1. **Overhead di HTTP/2:** In modalità polling, gRPC deve inizializzare un frame HTTP/2 e gestire i metadati dei Protocol Buffers per ogni singola richiesta. Per un payload di pochi byte, questo lavoro di "impacchettamento" è più oneroso rispetto alla semplicità di una stringa JSON piatta inviata via HTTP/1.1.

2. **Semplicità del dato:** Quando il volume informativo è minimo, il costo di parsing del JSON è trascurabile, rendendo la complessità della serializzazione binaria un inutile sovraccarico per la CPU.

5.2 Efficienza nei carichi complessi e Streaming

Il quadro cambia radicalmente non appena aumenta la complessità del dato o si passa alla modalità **Streaming**:

- **Payload Medium (Guadagno 70%):** In streaming, gRPC raggiunge l'efficienza massima. Dovendo gestire 200 stringhe, il marshaller binario scrive i dati direttamente nel buffer di rete, mentre REST deve generare una stringa JSON massiva includendo migliaia di delimitatori testuali (virgolette, virgole). Qui gRPC libera la CPU da un carico di formattazione oneroso.

- **Payload Large e Nested:** Nonostante il payload *Nested* costringa gRPC a una gestione ricorsiva più complessa, il guadagno rimane costante intorno al **30%**. In modalità streaming, gRPC annulla i tempi morti di setup, permettendo al marshaller di operare in un flusso continuo.

I dati dimostrano che gRPC non è la scelta ottimale per messaggi isolati di dimensioni minime (Polling Small), dove la semplicità di REST prevale. Tuttavia, per sistemi di telemetria che richiedono **scalabilità** (alto numero di sensori) o **complessità dei dati** (Medium/Large/Nested), gRPC diventa imbattibile, riducendo il carico computazionale fino al **71%** e prevenendo colli di bottiglia lato CPU.